

The Sage Programming Language

A Comprehensive Guide

Jacob Yates

August 2026 • Version 4.2.2 • Spec 2.0 • MIT License

Python-inspired, indentation-based systems language in C • Ten execution backends • Three GC modes • Self-hosted interpreter & compiler at full parity • Vulkan + OpenGL graphics • Agent AI, blockchain, ML/LLM, bare-metal OSdev

Synthesized from core/docs/*.md, core/docs/meta/PARITY.md, core/docs/meta/ROADMAP.md, core/docs/meta/CHANGELOG.md and README.md (v4.2.2). Repository: github.com/Night-Traders-Dev/SageLang

CURRENT STATUS • 18/18 phases complete • 2070+ tests • 10 backends • Self-hosted compiler 28/28 parity

Contents

Executive Summary	3
Part 1: Language Overview and Design Philosophy	3
1.1 Design Goals and Target Use Case	
1.2 Language Characteristics (v4.2.2)	
1.3 Execution Model (10 backends + profiles)	
1.4 Performance & Recent Updates v4.2.2-v4.1.0	
Part 2: Internal Architecture and Core Modules	5
2.1 Module Dependency Graph	
2.2 Lexer • 2.3 Parser • 2.4 Value System • 2.5 Environment	
2.6 Garbage Collector (Tracing / ARC / ORC)	
2.7 Interpreter • 2.8 Module System	
Part 3: Language Features by Phase (1-18)	9
Part 4: Writing Sage Programs	13
4.1 Basic Syntax • 4.2 Control Flow • 4.3 Functions & Closures	
4.4 Structs / Enums / Traits / Classes • 4.5 Exceptions	
4.6 Generators • 4.7 Modules • 4.8 Defer • 4.9 Match/Case	
Part 5: Implementation Details & Runtime Behavior	20
Part 6: Compilation and Execution	23
6.1 Building • 6.2 Entry Point • 6.3 Execution Flow • 6.4 Backends & Profiles	
Part 7: Advanced Topics and Internals	27
Part 8: Future Directions & Design Notes	28
Part 9: Bundled Library Modules (lib/)	29
Part 10: Native Standard Library (Phase 11)	34
Part 11: Concurrency and Async/Await	36
Part 12: Developer Tooling (Phase 12)	37
Part 13: Self-Hosting — Full Parity 28/28 (v4.2.0)	39
Part 14: Networking (Phase 14)	41
Part 15: JSON Library (cJSON Port)	42
GPU Graphics: Vulkan + OpenGL (Phase 15-16)	43
Library Support Matrix v4.2.2 (295 modules)	46
Appendix: Quick Reference	47

This PDF is generated from the canonical markdown sources. For live charts run: `make charts`, `python3 scripts/generate_backend_chart.py`, `python3 scripts/benchmark_recipes.py`. All 2070+ tests: `make test-all`, `make test-selfhost`.

SageLang (Sage) is a Python-inspired, indentation-based systems programming language written in C (C11) and self-hosted in Sage itself. It combines indentation-based blocks, dynamic typing, and first-class procedures/closures with low-level systems capabilities: manual memory primitives, FFI, inline assembly (x86-64/aarch64/rv64/mips/AVR), C-struct interop, and freestanding bare-metal/UEFI outputs. As of v4.2.2 the self-hosted Sage compiler produces byte-identical output to the C compiler across the full 28-case differential parity harness (C interpreter vs self-hosted interpreter vs self-hosted compiled binaries). The language supports ten execution backends, three GC modes, Vulkan + OpenGL graphics, agent/blockchain/ML/LLM stacks, and runs on desktop, Android/Kotlin, RP2040, and freestanding targets.

Spec version 2.0 is locked. All 18 development phases are complete (core language through ML/training, Linux kernel support, graphics, and self-hosting). This guide is synthesized from SageLang_Guide.md, SageLang_Reference.md, Import_Semantics.md, GC_Guide.md, Safety_Guide.md, JIT_AOT_Guide.md, Library_Support.md, Tooling_Guide.md, Concurrency_Guide.md, Vulkan_GPU_Guide.md, Baremetal_OSDev_UEFI_Guide.md, Networking_Guide.md, Blockchain.md, Cryptography_Guide.md, LLM_Guide.md, ML_CUDA_Guide.md, Discord_Bot_Guide.md, Agent_Chat_Guide.md, Android_Guide.md, FAT_FileSystem_Guide.md, SGVM_Guide.md, StdLib_Guide.md, Build_Guide.md, CLI_Reference.md, Self_Hosting_Guide.md, plus meta/PARITY.md, meta/ROADMAP.md, meta/CHANGELOG.md and README.md.

What is new in v4.2.2 (and catch-up from v4.2.1 → v4.1.0)

v4.2.2 — Standard Library Hardening & RWLock / Syscalls Expansion (current)

- **Stdlib parity:** Full alignment of `std.rwlock`, `os.linux.syscalls`, `metal.vga`, `metal.core`, `dicts` standard-library modules.

- **metal.core:** New hardware memory-barrier primitives in metal.core: dmb, dsb, isb, fence; CPU relax hints; cpu_id core-ID query; critical_section_enter/exit helpers; spin locks.
- **Hardening:** VGA text-mode already landed in 4.2.1 and now hardened: tightened string-repetition runtime helpers, validated command helpers (is_safe_command) and AOT process execution hardening (fork/execv/waitpid) against CWE-78 / CWE-400 / CWE-190.
- **Perf & safety:** String repetition and dict @inline annotations unified; SAGE_MAX_READ_SIZE 100MB consistently enforced (io.readfile, tcp.recv, etc.).

v4.2.1 — VGA Primitives & Hardware Timer Tracking

- **VGA/timer:** metal.vga: make_attr, putchar_at, puts, read_char_at, read_attr_at, draw_progress_bar; hardware timer mode state timer_get_mode; sigprocmask + signal mask constants.

v4.2.0 — Full Self-Hosted Compiler Parity (milestone)

- **Parity:** 28/28 parity across all three stacks (C interpreter, self-hosted interpreter, self-hosted compiled binaries). Closed: short-circuit and/or emission, full emitted-C runtime helpers (string slice/index/contains, chr/ord/int/indexof/type, bytes API, array concat, catchable div/mod, dict for-in, default params), first-class functions with compile-time arity guards, match guards, eager generators, full closure capture (locals promoted into per-invocation heap envs carried by closure values, nested named procs hoisted via two-pass discovery). Diagnostics now match C host.

v4.1.16 → v4.1.8 catch-up (still current in 4.2.2)

- **Stack guard:** v4.1.16 true stack-proximity guard (stack_danger() per-thread TLS, RLIMIT_STACK 75% budget, sage_raise_stack_limit 512MB headroom) — eliminates segfaults at ~280 frames debug / self-hosted double-interpretation edge; verified under AddressSanitizer.
- **Types:** v4.1.14-15 proc/let type annotations proc f(x: Int): Int fully in self-hosted parser/typecheck (TypeMap.declared, annotation_to_kind).
- **Backend fixes:** v4.1.11 AOT string interner sage_string_const (140MB/s leak fixed); v4.1.10-09 rich/SDL fixes; v4.1.8 C-backend short-circuit fix sage_bool(sage_truthy(L) && sage_truthy(R)).
- **Targets:** v4.1.7 AVR two-pass assembler for ATmega328P/B (avr_assembler.sage, avr_opcodes.sage, avr_hex.sage) → .hex flash; v4.1.1 AOT class codegen (s_ClassName, s_self, dispatch tables, @inline, val_native dedupe).

Part 1 Language Overview and Design Philosophy

1.1 Design Goals and Target Use Case

- Bridges Python and C: indentation syntax, dynamic typing, minimal C footprint (RP2040/RP2350).
- Systems programming: explicit GC control, class OOP, tagged Value model, FFI, raw memory, inline asm, freestanding outputs.
- Phases incrementally (1-18) without breaking prior functionality; correctness via mark-sweep/ARC/ORC, lexical Env chains, exception propagation.
- Opt-in safety: decoupled static analysis pass (safety.c / safety.sage) — @safe doc-comments + --strict-safety flag + unsafe: ... end barriers; ownership/borrow/lifetime/Option/ Send/Sync checks (Safety_Guide.md).

1.2 Language Characteristics (v4.2.2)

Feature	Details
Syntax	Python-like: indentation blocks, : terminators, soft keywords let/var/proc/class/struct/enum/trait/end/match/init (usable as identifiers in expressions)
Type system	Dynamically typed, runtime Value tags; v2.0 type annotations let x: Int, proc f(): Int validated by sage check /
Memory	Tracing (concurrent tri-color SATB, <1ms STW) default; ARC (--gc:arc); ORC (--gc:orc, Lins trial deletion). SAGE_MAX_READ_SIZE 100 MB
OOP	Classes single inheritance, self first param, super.init() auto-self, __str__/_eq_ dunder, struct/enum/trait native decls, -> alias for .
Control flow	if/elif/else, while, for...in (array/dict), break/continue, try/catch/finally/raise, yield/next, defer (LIFO), match/case/default with guards
Data	Arrays dynamic, dicts string-keyed hash (FNV-1a, linear probe, 75% grow, backward-shift delete), tuples immutable, bytes binary-safe, slices
Functions	First-class proc with closures, anonymous proc(x): body end, default params, generics proc f[T](x: T)->T, comptime/macro/quote/unquote, pragmas @inline etc.
Stdlib	110+ builtins + 295 lib modules (281/295 import clean on sage-c, 273/295 on sage); std/*, net/*, crypto/*, ml/*, llm/*, qpu/*, os/* (44+ parsers), blockchain/*, agent/*
Networking	POSIX socket, tcp, http (libcurl), ssl (OpenSSL); companion pure-Sage net/*, sage, discord gateway/REST

Concurrency	thread.spawn/join/mutex, async/await (thread-backed), atomics (<code>__atomic</code>), POSIX semaphores, condvars, rwlocks, SMP detection + affinity
GPU	Vulkan handle-table (~100 fns) + OpenGL 4.5 backend; shared <code>gpu_api.c</code> ; LLVM GPU bridges (103 <code>sage_rt_gpu_*</code>) + 30 BC_OP_GPU *: 17 <code>lib/graphics/*</code> libs
Compilation	C (<code>--emit-c/--compile</code>), LLVM IR with GPU (<code>--emit-llvm/--compile-llvm</code>), native asm x86-64/aarch64/rv64/mips/AVR, bytecode/SGVM, SageMetal, IIT/AOT/IIT+AOT, Kotlin/Android
Targets	<code>--compile-pico</code> (RP2040 uf2), <code>--compile-bare</code> (freestanding ELF), <code>--compile-uefi</code> (PE/COFF), profile suffixes <code>-baremetal/-osdev/-uefi</code>

LLVM codegen optimization: from module import CONST for foldable top-level lets (numbers/strings/booleans/nil) resolved at compile time including as aliases; C LLVM backend also resolves GPU constants and emits direct `sgpu_*` bridges; bytecode VM has 30 GPU opcodes for hot frame loops.

1.3 Execution Model

Shared front-end: source → lexer (indent stack, bracket-aware suppression) → parser (recursive descent, 12 precedence levels) → AST → one of ten backends. All modes share the same heap/GC/env model: global Env, child Envs per block/function/class, tagged Value (immediate numbers/booleans or GC-managed heap values).

```
Source -> Lexer (INDENT/DEDENT, 54 token types) -> Parser (12 levels) -> AST
-> AST interpreter (default) -> Bytecode compiler+VM (hybrid fallback via BC_OP_EXEC_AST_STMT)
-> SGVM binary (--sgvm) -> Lily transpiler (--compile-to-lily / --compile-from-lily)
-> C codegen (--emit-c / --compile) -> LLVM IR (--emit-llvm / --compile-llvm, GPU aware)
-> Native asm (--emit-asm / --compile-native, x86-64/aarch64/rv64/mips/AVR)
-> Freestanding ELF (--compile-bare) -> UEFI PE (--compile-uefi) -> SageMetal VM (make metal-vm)
-> JIT (--jit, profile + self-extract) -> AOT (--aot) -> JIT+AOT (--aot --jit)
-> Kotlin/Android (--emit-kotlin / --compile-android)
-> Runtime values in GC heap (tracing/ARC/ORC)
```

Hybrid bytecode: expressions/variables/loops/ calls go to stack bytecode; unsupported constructs (classes/imports/exceptions/generators) fall back to AST via `BC_OP_EXEC_AST_STMT`. Enable with `sage --runtime bytecode|auto`. LLVM-compiled GPU programs link `gpu_api.o` automatically (`-lvulkan -lglfw -lgl`) and use from `gpu` import CONST compile-time folding.

1.4 Performance Characteristics

Benchmark suite `benchmarks/01_fibonacci.sage .. 10_primes_sieve.sage` with paired Python 3 impls. Regenerate charts with `python3 scripts/generate_backend_chart.py`, `python3 scripts/benchmark_recipes.py`, `benchmarks/run_backend_compare.sh`; or make `benchmark-python`.

Workload	Python 3	Sage C	Sage LLVM	Sage AOT	Note
01 fibonacci	29.47 ms	0.81 ms (36x)	0.82 ms	0.77 ms (38x)	Compiled dominates
02 loop sum large	25.10 ms	0.78 ms (32x)	0.78 ms	0.76 ms (33x)	Tight loop
07 nested loops	180.59 ms	0.76 ms (238x)	0.78 ms	0.76 ms (238x)	C/LLVM huge win
AST vs CPython	—	—	—	—	AST ~ parity; VM faster on hot loops
AOT+JIT (micro)	—	AOT 1.96s	AOT+JIT 0.24s	—	Fibonacci(36); nested 5Kx5K AOT+JIT 0.14s

From README charts (12 workloads): hatched bars are codegen/emit throughput only. Self-Hosted double-interpretation cost is genuine tree-walking overhead, not a defect. Recipe benchmarks: run `make benchmark-python / make benchmark-python-md`. Backend comparison: Execution vs Codegen/emit-throughput sections independently scaled.

Part 2 Internal Architecture and Core Modules

2.1 Module Dependency Graph

```

main.c
|- lexer.c/h      [indentation, DOC_COMMENT, 54 token types]
|- parser.c/h     [recursive descent, 18 expr + 22 stmt types]
|- value.c/h      [tagged Value union, 19+ types]
|- env.c/h        [lexical Env chain, GC-marked]
|- gc.c/h         [tracing/ARC/ORC, gc_pin/unpin, SATB barrier, 3 GC modes]
|- ast.c/h        [node factories]
|- token.h        [TokenType 54]
|- module.c/h     [module cache, search . / lib / modules, realpath containment]
|- compiler.c     [C backend, runtime helpers, sage_string_const interner]
|- llvm_backend.c [LLVM IR + GPU const resolution + gpu call dispatch]
|- llvm_runtime.c [103 sage_rt_gpu_* bridges, 644->1808 lines]
|- codegen.c/h    [native asm x86-64/aarch64/rv64/mips, x-target profiles]
|- graphics.c/h + gpu_api.c/h [Vulkan + OpenGL, handle-table]
`- src/vm/ bytecode.c/vm.c/program.c/runtime.c [30 GPU opcodes]

```

2.2 Lexer (lexer.c/h)

Tokenizes with indent stack (INDENT/DEDENT). 47 keywords including soft keywords `match/init/enum/struct/trait/end/print` (usable as identifiers in `expr` assignments). Operators: `+*/% == != < > <= >=` and/or/not in `& | ^ ~ << >>` and unary `- ~` not. Structural: `INDENT/DEDENT/NEWLINE/DOC_COMMENT/EOF/ERROR`. Enforces `SAGE_MAX_READ_SIZE` 100MB on I/O and network `recv`; bracket-aware suppression inside `{}` `[]` `()`; max identifier 1024, max string literal 4096.

2.3 Parser (parser.c)

Recursive descent, 12 precedence levels (or `<` and `<== < < >` in `< | < ^ < & < << >> < ++ < */% < unary < postfix . ->` `[]` `()`). Statements: `LET/PRINT/IF/WHILE/FOR/BLOCK/PROC/CLASS/RETURN/BREAK/CONTINUE/TRY/RAISE/YIELD/DEFER/MATCH/IMPORT/ASYNOC/STRUCT/ENUM/TRAIT/COMPTIME/MACRO_DEF`. Expressions: 18 types (NUMBER/STRING/BOOL/NIL/BINARY/VARIABLE/CALL/ARRAY/INDEX/DICT/TUPLE/Slice/GET/SET/INDEX_SET/AWAIT/SUPER/COMPTIME). Specifics: v4.1.3 soft-keyword handling, v4.1.8 short-circuit and/or fix emitting `sage_bool(sage_truthy(L) && sage_truthy(R))`.

2.4 Value System (value.h/c)

Tagged union Value { ValueType type; union { double number; int boolean; char* string; NativeFn native; FunctionValue* function; ArrayValue* array; DictValue* dict; TupleValue* tuple; ClassValue* class_val; InstanceValue* instance; ExceptionValue* exception; GeneratorValue* generator; CLibValue* clib; PointerValue* pointer; ThreadValue* thread; MutexValue* mutex; } }. Heap types prepend `GCHeader { marked, type, next }`.

Type	Storage	Mutable	Example
VAL_NUMBER	double (stack)	Yes	10. 3.14. 0xFF. 0o755. 0b1010
VAL_BOOL	int	Yes	true / false
VAL_NIL	—	No	nil (Option[T] in @safe)
VAL_STRING	char* heap	No (immutable)	"hello" \n \t \xHH
VAL_ARRAY	ArrayValue*	Yes	[1,2,3]. push/pop. slice. repeat
VAL_DICT	DictValue* (hash)	Yes	{"k": v}. dict set/has/delete/keys/values
VAL_TUPLE	TupleValue*	No	(1,2,3)
VAL_FUNCTION	FunctionValue* closure	No	proc f(x): ... end, proc(x): body end
VAL_NATIVE	C fn ptr	No	len, print, etc. (84 globals)
VAL_CLASS/INSTANCE	ClassValue/InstanceValue*	—	class C: / C() ; fields via dict
VAL_EXCEPTION	ExceptionValue*	No	raise "msg" (string/num/bool/nil)
VAL_GENERATOR	GeneratorValue*	Yes	yield/next protocol
VAL_BYTES	BytesValue*	Yes	bytes(). binary-safe
VAL_POINTER/CLIB/THREAD/MUTEX	*	—	mem_alloc / ffi_open / thread.spawn / mutex

Dicts: FNV-1a open-addressing linear probing, 75% grow, backward-shift deletion (no tombstones). Strings immutable; arrays/dicts by reference. v4.2.2 hardened `string_repeat/array_repeat` helpers with overflow/length guards.

2.5 Environment and Scoping (env.h/c)

Env { EnvNode* head; Env* parent; Env* alloc_next; int marked; } + EnvNode { char* name; Value value; next; }. Lexical

chain: env_define (current scope only, update if exists), env_get (walk parents), env_assign (walk chain to reassign). SATB write barrier on env_define/assign old_value. GC: env_sweep_unmarked frees unreachable Envs, marks cleared during sweep (no separate pass), env_clear_marks mutex-protected as of 16d audit. Call stack: each proc/class/block creates child Env; closures capture defining Env.

2.6 Garbage Collector (gc.h/c) — 3 modes, hardened in 4.2.2

Tracing (default) — concurrent tri-color mark-sweep + SATB

Four phases: ROOT SCAN (STW 50–200us, snapshot roots, shade GRAY, enable barrier) → CONCURRENT MARK (drain gray stack, children GRAY, parent BLACK, mutator runs, allocated BLACK) → REMARK (STW 20–50us, re-scan roots, drain barrier, disable) → CONCURRENT SWEEP (free WHITE in 256/batch, reset survivors to WHITE). Invariant: no BLACK → WHITE. Trigger: object count ≥ next_gc_objects OR live bytes ≥ next_gc_bytes (adaptive padding: <12.5% reclaim → shrink, >100% → grow). Root set: global Env chain, function registry, call stack, thread registry (TLS roots). Stats: gc_stats()[bytes_allocated, num_objects, collections, objects_freed, next_gc] and gc_enable_debug() per-phase timing.

```
GC_WRITE_BARRIER(old_value); slot = new_value; // shades old gray if white
// Insertion points: env_define, env_assign, array_set, dict_set, instance_set_field
// Pinning: gc_pin()/gc_unpin() around multi-step allocations
// Marking now covers VAL_CLIB/POINTER/THREAD/MUTEX (16d audit fix)
```

ARC (--gc:arc)

Deterministic reference counting; immediate free when refcount→0; simple cycle buffer (every 1000 decrements). Best for linear graphs/real-time.

ORC (--gc:orc) — recommended for complex graphs

Nim-inspired: ARC base + Lins trial deletion three-phase cycle collector (PURPLE candidate marking on decrement>0 → scan trial-decrement → collect WHITE). Trigger every 500 decrements. Colors: BLACK normal, PURPLE candidate, GRAY scanning, WHITE confirmed garbage. API: --gc:tracing|arc|orc or gc_set_arc()/gc_set_orc(), gc_mode(). All modes share mutex-protected GC, TLS root moves, AST GC push/pop guards, and gc_track_external_free on exceptions.

2.7 Interpreter (interpreter.c/h)

Tree-walking with ExecResult {value, is_returning/breaking/continuing/throwing/yielding, exception_value, next_stmt}. Statement dispatch: STMT_LET/EXPRESSION/PRINT/IF/WHILE/FOR(proc+range/array/dict)/BLOCK(child Env + defer stack 64/1024 slots)/PROC registry/CLASS/methods+inheritance/RETURN/BREAK/CONTINUE/TRY (catch + finally override semantics)/RAISE (any value→string message)/YIELD (generator state)/IMPORT (module cache + realpath containment). Expression eval: literals, variables (env_get), binary (short-circuit and/or in interpreter + C backend), calls (child Env + param bind), indexing/slicing (negative wrap for strings), property get/set via dict, super auto-self. Generator protocol: body_has_yield detection, run_generator eager collection in self-hosted, next(gen) resumes. Native registry 84 globals + module-scoped io/sys/math/strings/json etc.; includes val_tag/build_info, type/chr/ord/hash/sizeof/doc, gc_*, next, ffi_*, mem_*, asm_*, struct_*, atomics/semaphores, path/*, hash/sha256, bytes_*, struct_def etc. Hardening: stack_danger() TLS proximity guard (75% RLIMIT_STACK budget, 512MB headroom), MAX_RECURSION 1e6 stmts, MAX_LOOP 1e6 iterations, SAGE_MAX_READ_SIZE, recursion/parser depth limits, shift 0–63 validation, FFI max 3 args, negative-offset rejection.

2.8 Module System (module.h/c, Import_Semantics.md)

Module {name, path, env, is_loaded/loading, next}. ModuleCache {modules linked list, search_paths}. Lifecycle load_module(cache, "math") searches ./name.sage then ./name/_init_.sage under search_paths . / ./lib / ./modules (+ SAGE_LIB_PATH env) → create Module → execute_module in module Env (child of global) → cache. Import syntax: import math, import math as m, from math import sin, cos, from math import sin as sine, from X import * (star), dotted imports os.fat / os.linux.syscalls. Search order: ./relative, lib/, SAGE_LIB_PATH. Export: all top-level lets/procs are visible (no export keyword). Caching: load once per session; is_loading flag prevents cycles. Path security: module name validation rejecting / \ .. plus realpath containment. Compiled LLVM const import optimization: from module import CONST resolved at codegen for foldable scalars including GPU constants (sgpu_* bridge).

Part 3 Language Features by Phase (1-18 complete)

Phase 1-3: Core Language Arithmetic, variables, control flow, functions. Example: let x=10; proc add(a,b): return a+b; if/while blocks.

Phase 4: Garbage Collection Mark-sweep now tri-color concurrent (tracing/ARC/ORC). Manual gc_collect(), gc_stats(), gc_enable/disable, gc_mode() switching.

- Phase 5-6: Data Structures & OOP** Arrays ([1,2,3], push/pop, slice arr[1:3], arr[2:]), dicts ({"k":v}, dict_has/keys/values/delete), tuples (1,2,3) immutable. Classes: class Animal: proc init(name): self.name=name; class Dog(Animal): super.init(name); method override, self binding, inheritance chain.
- Phase 7: Control Flow, Exceptions, Generators** try/catch e / finally (finally return/break/raise overrides try/catch — Python/Java semantics); raise any value → string message; generators yield/next with resumable state; defer LIFO (64/1024 slots); match/case/default with if-guard syntax case 42 if y>0:.
- Phase 8: Modules** Import alias/star/dotted-path; search + caching + cycle detection; VAL_MODULE dot-access; realpath containment.
- Phase 9: Low-Level Programming** Bitwise & | ^ ~ << >> (shift 0-63 else 0, arithmetic right-shift, float→long long truncation); ffi_open/call(lib, func, ret, ... up to 3 args)/sym/close (GC-tracked); mem_alloc (cap 64MB, zeroed)/free/read(offset+type)/write/size/addressof (bounds + double-free guards); asm_exec/compile/arch (x86-64/aarch64/rv64/mips + AVR assembler); struct_def/new/get/set/size with C ABI alignment; unsafe: ... end blocks.
- Phase 10: Compiler Pipeline** C codegen (scalar control flow→arrays), for-in, dicts/tuples/exceptions/properties/strings/memory/struct, LLVM IR (GPU aware), native asm, passes typecheck/constfold (-O1)/DCE (-O2)/inline (-O3).
- Phase 11: Native Stdlib + Concurrency** math/io/string/sys/socket/tcp/http/ssl natives via create_native_module; threading thread.spawn/join/mutex/lock/unlock/sleep/id + GC thread safety (pthread mutex + TLS roots); async proc + await (is_async flag, thread_spawn_native).
- Phase 12: Tooling Ecosystem** REPL multi-line + 22 commands; formatter sage fmt / fmt --check; linter 13 rules E001-003/W001-005/S001-005; TextMate + VSCode syntax; LSP sage --lsp / sage-lsp diagnostics/completion/hover (g_hover_docs)/formatting.
- Phase 13: Self-Hosting** Lexer (~300 lines) parser (~700, 12 levels) interpreter (~1050, dict-based Value) + token.sage/ast.sage/sage.sage/typecheck.sage (228 lines). Bootstrap coverage: arithmetic/vars/control/functions/closures/classes/arrays/dicts/strings/exceptions/generators/match/defer/biwise NOT/modules/type-annotations soft-keywords (match/init/enum/struct/trait/end) + loop limits. Full parity 28/28 as of 4.2.0.
- Phase 14: Networking & Data Interchange** socket (15 fns + AF/SOCK/IPPROTO constants) / tcp (9 fns) / http (libcurl get/post/put/delete/patch/head/download/escape) / ssl (OpenSSL 13 fns) + cJSON port lib/json.sage (~1050 lines, 13 creation/10 type-check/modification + ToSage/FromSage, 88 tests).
- Phase 15: Vulkan Graphics Library** 3-layer handle-table Vulkan (~2600 lines, 100+ fns) → lib/vulkan.sage builders → lib/gpu.sage; instance→device→queues→buffers/images/samplers/shaders/descriptors/pipelines/render-passes/command-pools/sync; 100+ constants; 9 demos; 104 GPU tests.
- Phase 16: LLVM GPU + OpenGL + Audits** gpu_api.h/c pure C API; 103 sage_rt_gpu_* LLVM bridges; 30 BC_OP_GPU_* hot-path opcodes; OpenGL 4.5 alt backend (SAGE_HAS_OPENGL, opengl.sage); audits: break/continue in bytecode (loop context stack, 10 new opcodes), VINST_BRANCH for asm 3 archs, GC VAL_CLIB/POINTER/THREAD/Mutex cleanup (16d), finally override, raise non-string conversion, shift/FFI/mem guards.
- Phase 17: Backpropagation, cuBLAS, NPU** Backprop 1-layer SwiGLU, ml_native.train_step/forward_pass/load_weights, C trainer 19+ steps/sec, cuBLAS FP32 auto-detect, ARM NEON/RISC-V Vector, Adam+cosine+clip, SL-TQ-LLM d=96 197K params, NPU backends (Hexagon/Exynos/NNAPI/NEON/RVV/ONNX), TurboQuant 3-bit KV 8.5x, AutoResearch, GGUF import.
- Phase 18: Linux Kernel Support + QEMU** 11 Linux modules syscalls/driver/kmodule/procfs/netlink/sysfs/devicetree/cgroups/epoll/ioctl/namespace (multi-arch), plus os/* 44+ parsers, paging/idt/serial/dtb/alloc/vfs/ext/btrfs/f2fs/boot/gdt/linker/kmain/console/keyboard/time r/syscall/pmm/vmm/diskimg/iso/qemu; single VERSION source.
- Sentinel/Veritas hardening (4.2.x, 4.1.7-4.1.3)** AOT buffer overflow→dynamic alloc, Bolt inline caching ~27% loop speedup (~SAGE_ALLOC reorg), sentineled GC pinning, threaded VM computed gotos, SageMetal freestanding VM, gas metering vm_gas_* , IRQ double-reg guards, Vulkan temp-file CWE-377 fix, algorithmic DOS fixes (crypto to_hex O(N^2)→O(N), sandbox code extraction), binary-safe I/O (null-byte aware interning).

Part 4 Writing SageLang Programs

4.1 Basic Syntax Patterns

```
let x = 42; var y = 100; let s = "Hello\nWorld\x41"; let b = true; let n = nil
let arr = [1,2,3]; let d = {"a": 1, "b": 2}; let t = (1, "two", 3)
print 10 + 5    # 15    5 & 3 == 1    5 | 3 == 7    ~0 == -1    1 << 4 == 16
"a" + "b"      # "ab"    len("hi") == 2    "a,b".split(",") == ["a","b"]
push(arr, 60); arr[0] = 100; arr[1:3] # slice
# literals: 42  3.14  0xFF  0o755  0b1010  "str"  true/false  nil
```

Variables `let`/`var` both reassignable in spec 2.0 (`let` conventionally immutable, not enforced). Type annotations: `let x: Int = 42`; `let s: String = "hi"`; `let xs: Array[Int] = [1,2]`; `proc add(a: Int,b: Int)->Int`:. Optional types `T?`. Doc comments `##` above procs retrievable via `doc(fn)`. Lifetimes checked under `--strict-safety / @safe`.

4.2 Control Flow Patterns

```
if x < 10:
    print "small"
elif x < 20:
    print "medium"
else:
    print "large"

for item in [10,20,30]:
    print item
for i in range(0,5):
    if i==3: continue
    if i==7: break
    print i

let i=0; while i<5:
    print i; i=i+1
```

for iterates arrays/tuples/dicts (dict → insertion-order keys); strings not directly iterable — use `while i < len(s): s[i]` or `range(len(s))`. Recursion/loop/loop-nesting caps: 1e6 stmts, 1e6 loop iterations, 1024 nesting levels.

4.3 Functions and Closures

```
proc greet(name):
    print "Hello, " + name
proc add(a,b): return a+b
proc make_multiplier(factor):
    proc multiply(x): return x * factor # captures factor
    return multiply
let times3 = make_multiplier(3); print times3(10) # 30
# anonymous: let f = proc(x): return x*2 end; default params: proc connect(host, port=8080):
```

Closures capture defining Env (FunctionValue* closure). Recursion allowed; small non-recursive single-return procs inlined at -O3. In self-hosted emitted C, capturing parents promote locals into per-call heap envs with hidden `_cenv` (28/28 parity fix). Max identifier 1024, max string literal 4096.

4.4 Structs, Enums, Traits, and Classes

```

struct Point:
  x
  y
end
let p = Point(10,20); print p.x # 10; Point(x=10,y=20)
enum Color:
  Red
  Green
  Blue
print Color.Red # 0 (dict-mapped)
trait Printable:
  proc to_string(self)
end
class Point2:
  proc init(x,y): self.x=x; self.y=y
  proc distance(): return self.x*self.x + self.y*self.y
class Dog(Animal):
  proc init(self, name, breed): super.init(name); self.breed=breed
  proc speak(self): super.speak(); print self.name + " barks"
let d = Dog("Rex","Labrador"); d.speak() # super->init also works
let q = Point(3,7); print q->x # -> alias for . (field read/write/method/super)

```

Native struct/enum/trait lowered to class/instance/dict at runtime (VAL_CLASS/INSTANCE). Soft keywords: these decl keywords also usable as variable names in expressions. Super: inherited method dispatch with auto-self injection (3+ levels).

4.5 Exception Handling

```

proc safe_divide(a,b):
  if b==0: raise "Zero denominator"
  return a/b
try:
  print safe_divide(10,0)
catch error:
  print "Error: " + error
finally:
  print "Done"
# raise 404 -> "404" ; raise nil -> "nil" ; finally return/break/raise overrides try/catch

```

4.6 Generators

```

proc fibonacci(n):
  let a=0; let b=1; let c=0
  while c<n:
    yield a
    let tmp=a+b; a=b; b=tmp; c=c+1
let gen = fibonacci(5)
print next(gen) # 0 1 1 2 3 then nil
# call-site eager collection in self-hosted emitted backend (SAGE_TAG_GENERATOR per-proc collector)

```

4.7 Modules

```

# math.sage: proc square(x): return x*x
import math as m; print m.square(5)
from math import sin, cos; from math import sin as sine
import os.fat; import os.fat as fat_module
from os.fat import parse_boot_sector # dotted path

```

Search: ./file.sage, lib/, SAGE_LIB_PATH; directory packages via __init__.sage. Module caching; is_loading cycle guard; realpath containment for / \ .. validation. Compiled LLVM resolves from module import CONST at codegen (numbers/strings/bools/nil plus foldable exprs).

4.8 Defer Statements (LIFO)

```

proc process_file(path):
  let h = open(path)
  defer:
    close(h); print "closed"
  # work ... defer runs on normal/return/break/continue/exception exit
proc multi():
  defer: print "third"
  defer: print "second"
  defer: print "first"
  print "working" # working, first, second, third

```

Interpreter: 1024-slot defer stack per block; compiled path: partitioned scope + emitter defer-scope stack; both stacks at parity (defer semantics ported in parity audit).

4.9 Pattern Matching (match/case/default)

```

match cmd:
  case "quit": print "quitting"
  case "hello": print "greeting"
  case 42 if y>0: print "forty-two positive" # guard (parser stores clause guard)
  default: print "unknown"
# lowered to if/else-if via sage_equal (C) / sage_rt_eq (LLVM) / native isel

```

Match supports numbers/strings/bools/nil with value equality; guards are full expressions; all backends support it (interpreter/C/LLVM/native).

Part 5 Implementation Details & Runtime Behavior

5.1 Memory Layout and Object Representation

Stack (cheap): Value struct for numbers/bools/nil/native fns. Heap (GC): arrays/dicts/tuples/strings/classes/instances/exceptions/generators/clib/pointers/threads/mutexes — each with GCHdr (marked/type/next linked list). Strings immutable (new allocation on assign); arrays/dicts mutable passed by reference. OOM: SAGE_ALLOC/SAGE_REALLOC/SAGE_STRDUP abort on failure (never returns NULL). Pointer/Struct interop alignment follows C ABI natural alignment + tail padding.

5.2 Call Stack and Environments

Each call allocates a child Env (parent = current). Parameters bound via env_define; return pops with ExecResult.is_returning. Generator state retains Env + next_stmt resume point. Thread-safety: Env list ops & GC protected by env_mutex + global GC mutex; roots now in TLS (per-thread origins).

5.3 Exception Propagation

ExecResult.is_throwing carries exception_value up the stack until try/catch match; finally always executes; finally control flow (return/break/continue/raise) takes precedence (Python/Java parity). Non-string raise values coerced to string (numbers/bools/nil → message, else "Unknown error"); fixed to avoid double object-dump in C EXPR_GET error path (parity audit).

5.4 Generator State Management

Proc containing yield flagged; call returns VAL_GENERATOR with captured Env + bytecode offset. next(gen) re-enters interpreter at next_stmt until next yield or return (then exhausted → nil). Eager-collector variant for emitted C: per-proc collector fn gathers all yields under SAGE_TAG_GENERATOR; for-in rejection on generators normalized to C behavior across stacks.

5.5 Operator Precedence and Associativity (12 levels)

Level	Operators	Assoc
1	or	Left
2	and	Left
3	== !=	Left
4	< > <= >= in	Left

5	(bitwise OR)	Left
6	^ (XOR)	Left
7	& (AND)	Left
8	<< >>	Left
9	+ -	Left
10	* / %	Left
11	unary ~ not	Right
12	, -> [] ()	Left

5.6 Type Coercion and Truthiness

Sage uses `sage_truthy(v)` everywhere (also in C backend `sage_bool(sage_truthy(L) && sage_truthy(R))` for short-circuit and/or). Falsy: `nil`, `false`, `0`, `0.0`, `""` and empty collections per language spec; everything else truthy. Structural equality (`==`) deep-compares arrays/dicts; class instances reference-equal by default or via `__eq__` dunder. Type names via `type(v)` ("bytes" for byte buffers, etc.).

Part 6 Compilation and Execution

6.1 Building the Interpreter

Desktop link: `libm`, `pthread`, `dl`, `libcurl`, `OpenSSL`; auto Vulkan/GL via `pkg-config` (`SAGE_HAS_VULKAN` / `SAGE_HAS_OPENGL`).

```
make clean && make -j$(nproc)          # -> ./sage + ./sage-lsp
make DEBUG=1                          # -g -O0 -DDEBUG
make VULKAN=0 OPENGL=0                # disable GPU
cmake -B build && cmake --build build   # CMake default
cmake -B build -DBUILD_SAGE=ON && cmake --build build # self-hosted (sage_host)
./sagemake info; ./sagemake build; ./sagemake chatbot --llvm; ./sagemake train 200000 0.001
```

6.2 Main Entry Point (main.c)

Parses CLI (`sage --help table`), initializes GC (tracing default, flags `--gc:tracing|arc|orc`), detects GPU backends, selects runtime (`--runtime ast|bytecode|jit|aot|auto`), dispatches REPL vs file vs compile pipelines. Hardened: 100MB I/O caps, recursion guards, real C-stack guard init (`RLIMIT_STACK`), TLS root registration.

6.3 Program Execution Flow

```
CLI parse -> GC init -> lexical analysis (lexer) -> recursive-descent parse (AST)
-> safety pass (--strict-safety / @safe) -> optimization passes (run_passes):
    -01 constfold, -02 DCE, -03 inline + typecheck (-01+)
-> backend emit (C/LLVM/asm/VM/SGVM/Kotlin/JIT/AOT) OR direct interpretation
-> linking (cc -O2 -fno-strict-aliasing or clang -lvulkan -lglfw -lGL or aarch64-linux-gnu-gcc)
-> execution with GC heap + Env chain + signal-safe stack guard
```

6.4 Runtime Backends (10 + profiles)

Backend	Command	Notes
AST interpreter	<code>sage file.sage</code>	Tree-walking, full language, default
Bytecode VM	<code>sage --runtime bytecode file.sage</code>	Stack VM, hybrid fallback, 30 GPU opcodes
SGVM binary	<code>sage --sgvm file.sage</code>	Binary bytecode artifact via MetalVM
Lily transpiler	<code>sage --compile-to-lily / --compile-from-lily</code>	Interop with Lily language
C codegen	<code>sage --emit-c / --compile file.sage -o bin</code>	Generates C, cc -O2 -fno-strict-aliasing; short-circuit <code>sage_bool(...&&...)</code>
LLVM IR	<code>sage --emit-llvm / --compile-llvm</code>	.ll + gpu api.o link, GPU const folding
Native asm	<code>sage --emit-asm --target x86-64 aarch64 rv64 mips avr</code>	--target profiles: -baremetal/-osdev/-uefi
Freestanding ELF	<code>sage --compile-bare</code>	<code>sage</code> entry symbol, bare-metal kernel
UEFI PE	<code>sage --compile-uefi</code>	efi main symbol, EFI app
SageMetal VM	<code>make metal-vm</code>	Freestanding bytecode object, --emit-vm/svm
JIT	<code>sage --jit file.sage [-o bin]</code>	Profiling (100+ calls), tail-call trampolines, bundled deps, x86-64/AArch64/RV64
AOT	<code>sage --aot file.sage -o bin</code>	Type-specialized C11, class compilation, comptime inline, cross-compile via * gcc
JIT+AOT	<code>sage --aot --jit file.sage -o bin</code>	Profile-guided AOT (JIT feedback -> AOT specialization)

Kotlin/Android	sage --emit-kotlin / --compile-android	Kotlin sealed Value + Gradle project
----------------	--	--------------------------------------

Native asm profiles: hosted (default executable flow), -baremetal/-osdev (freestanding sage_entry object), -uefi (efi_main object/PE). Pico: --compile-pico -> .uf2 (RP2040/RP2350-arm/riscv via Pico SDK). AVR: core/boards/AVR two-pass assembler -> Intel HEX .hex (ATmega328P/B, blink example, encoder verified vs Microchip datasheet + simavr). Optimizations: constfold (2+3→5, constant branch elimination, 64KB concat limit), DCE (unreachable after return/break, unused lets/procs), inline (small non-recursive single-return). All passes updated for EXPR_AWAIT / STMT_ASYNC_PROC / match/defer types.

Part 7 Advanced Topics and Internals

7.1 Memory Leaks and GC Limitations

Fixed unbounded AOT leak (140MB/s) via sage_string_const interner + GC rooting (v4.1.11). Named leakers fixed: VAL_CLIB name strings, VAL_POINTER owned memory, VAL_THREAD handles, VAL_MUTEX destroy, exception message tracking via gc_track_external_free, Env GC guards (AST_GC_PUSH/POP). ARC/ORC give deterministic immediate free; tracing gives max throughput with <1ms pauses. Don't mix manual mem_alloc/free with GC heap — they are distinct arenas (mem_* GC-tracked but owned flag governs free).

7.2 Performance Characteristics (engineering)

Inline caching (Bolt) for EXPR_VARIABLE/EXPR_SET ~27% loop speedup (v4.1.3); threaded VM via computed gotos; register-pinned VM dispatch (state in registers, branch-predicted stack checks); JIT tail-call trampolines actually execute compiled functions (x86-64/AArch64/RV64); GPU paths use direct sgpu_* (no Value dispatch) for 30 hot opcodes; ORC trial deletion every 500 decrements; string helpers O(n) write-pointer (join/replace), URL parsing de-quadratic, Env marks O(1) flag (ex O(n^2) list scan).

7.3 Extending Sage with Native Functions

```
Value my_fn(int argc, Value* argv) {
    if (argc!=1 || argv[0].type!=VAL_NUMBER) return makeError("expected number");
    return makeNumber(argv[0].as.number * 2);
}
// register: env_define(g_global_env, "my_fn", 5, makeNative(my_fn));
// GC: use AST_GC_PUSH/POP around allocations; access with SAGE_AS_NUMBER etc.
```

7.4 Debugging and Introspection

REPL: :type, :ast, :env, :modules, :emit-c/llvm/kotlin, :stats (GC+stack+CPU), :time, :bench n expr, :doc name, :gc, :runtime. LSP hover uses shared g_hover_docs source of truth including user-defined procs. Builtins: val_tag(v) numeric tag, build_info()->{version, arch, build_type, spec_version}, doc(v), type(v), gc_stats(), gc_mode(), gc_collections(). Debug VM flag gc_enable_debug() prints root/mark/remark/sweep timing per collection (#42: root=150us mark=2300us remark=35us sweep=800us).

7.5 Interpreter Safety Limits (hardened)

Limit	Value	Enforcement
SAGE_MAX_READ_SIZE	100 MB	io.readfile/readbytes.tcp.recv (CWE-400)
Recursion (stmts)	1,000,000 + stack_danger() TLS guard	Catchable exception before OS stack overflow (75% RLIMIT_STACK, 512MB headroom)
Parser recursion	100,000	Error recovery in REPL
While iterations	1,000,000	Catchable exception per loop
Loop nesting	1,024	Compile error MAX_LOOP_DEPTH
String literal	4,096 chars	Lexer rejects
Identifier	1,024 chars	Lexer rejects
mem_alloc_cap	64 MB	mem_alloc hard error
Shift amount	0-63 else 0	Bitwise guard, arithmetic right-shift
FFI args	<=3	ffi_call hard error
mem_offset	>=0 + bounds checked	mem_read/write reject negative + overflow

Part 8 Future Directions and Design Notes

8.1 Planned Features (from ROADMAP)

All 18 phases complete per ROADMAP.md (June 9 2026 checkpoint → August 2026 4.2.2 hardening). Roadmap is now maintenance: growing stdlib breadth (295→), backend completeness, security audits, and graphics/ML model training. Open items tracked in core/docs/meta/PARITY.md: builtin completion (serialize/thread families, fat_*, probe/pack64/shell_exec, from-import-as of native values via stdlib registry) and remaining std.enum/trait gap in self-hosted parser. WebAssembly backend is future direction; native codegen module/class/GPU still landing (LLVM path already covers GPU). Package manager CLI/registry/version resolution proposed but not yet implemented.

8.2 Design Decisions and Rationale

Indentation kept for readability + C-level explicit heap control (manual mem_* alongside GC). Safety as decoupled opt-in pass keeps C/LLVM/native backends simple and explicit for OSdev/bare-metal. Hybrid bytecode preserves full language coverage while accelerating hot loops. Three-tier GPU design isolates pure C gpu_api from execution paths, letting LLVM/VM share ~100 sgpu_* implementations. Self-hosting proves compiler correctness and enables differential parity CI (run_parity.sh 28 cases). Type annotations validated at runtime by sage check / safety pass, not by backend specialization (AOT uses profile-guided or inference hints).

Part 9 Bundled Library Modules (lib/) — 295 modules

Import/behavior verified by importing all 295 modules from neutral directory (Library_Support.md v4.2.2). Fully supported under BOTH interpreters: arrays, assert, chat/*, crypto/*, dicts, iter, json, math, option, perf, rich/*, stats, std/* (except std.enum/trait on sage), strings, string, sys, utils, transpiler/lily/*, agent/*, blockchain/* (see exclusions), net/*, android/*, cuda/*, discord/*, gc/*, llm/*, metal/* (now including vga + core hardened), mips/*.

9.1 arrays • 9.2 strings • 9.3 dicts • 9.4 iter • 9.5 stats • 9.6 utils

arrays.sage:	map/filter/reduce/unique/zip/chunk/flatten,
array_contains/index_of/min/max/sum/product/extend/repeat/reverse.	strings.sage: contains/padding/repeat helpers.
dicts.sage: query helpers, fallback reads, @inline annotations hardened in 4.2.2.	iter.sage:
count/range_step/enumerate/cycle/take.	stats.sage: mean/variance/stddev/normalization.
default_if_nil/swap/head/last.	utils.sage:

9.7 assert • 9.8 math • 9.9 GPU Libraries • 9.10 OSdev Libraries

assert: rich assertions. math: pow_int/factorial/gcd/lcm/sqrt helpers. GPU: lib/gpu.sage high-level run_compute + ping-pong, lib/vulkan.sage + opengl.sage builders, plus 17 lib/graphics/* libs (see GPU chapter). OSdev: 44+ os/*.sage binary parsers/hardware abstractions (fat/fat_dir/elf/mbr/gpt/pe/pci/uefi/acpi/paging/idt/serial/dtb/alloc/vfs/ext/btrfs/f2fs/boot/start/multiboot/gdt/linker/kernel/kma in/console/keyboard/timer/syscall/pmm/vmm/diskimg/iso/qemu/linux/* 11 modules, smp topology). Bare-metal profiles emit sage_entry / efi_main.

9.11 Networking Libraries

net/* (Sage-level): url (parsing/percent-encoding), headers, request builder, server (TCP/HTTP), websocket (RFC6455, parse error both hosts pre-existing), mime (80+ types), dns (wire-format), ip (IPv4/CIDR), plus native socket/tcp/http/ssl.

9.12 Cryptography Libraries

crypto/hash (SHA-256/SHA-1/CRC-32, to_hex hardened O(N) in 4.0.4), crypto/hmac (RFC2104), crypto/encoding (Base64/hex), crypto/cipher (XOR/RC4/CBC/CTR), crypto/rand (xoshiro256** PRNG, UUID v4, Fisher-Yates), crypto/password (PBKDF2-HMAC).

9.13 Machine Learning Libraries

ml/tensor (N-dim shape), ml/nn (Linear/ReLU/Sigmoid), ml/optim (SGD/Adam + schedulers), ml/loss (MSE/cross-entropy/Huber/KL), ml/data (Dataset/DataLoader/batching/normalization), ml/debug (weight stats/grad check), ml/viz (SVG charts), ml/monitor (live training), ml/gpu_accel (CPU fallback — sage needs ml_native C accelerator; stub on self-hosted), ml/npv (Qualcomm/Exynos/NNAPI/NEON). Trainer: make train-c / train-sage, SwiGLU 1-layer backprop, SageGPT models/*.

9.14 CUDA Libraries

cuda/device (descriptors, arch detection), cuda/memory (GPU alloc + host/device transfers), cuda/kernel (1D/2D/3D launch), cuda/stream (streams/events/timing). Shielded behind SAGE_HAS_CUDA; cpu fallback path present.

9.15 Standard Library (lib/std/) — overview (see Part 10)

std/regex, datetime, log (TRACE-FATAL), argparse, compress (RLE/LZ77), process, unicode (UTF-8), fmt (hex/bin/oct/tables), testing (assertions), enum (Result/Option), trait, signal (pub-sub), db (tables/CRUD/joins), channel (Go-style buffered), threadpool (work queue/parallel map), atomic (ints/flags/spinlocks), rwlock (now 4.2.2 aligned — see Concurrency), condvar (barriers), debug, profiler (hierarchical), docgen, build (project config), interop (FFI struct pack/unpack).

9.16 LLM / Neural Network Libraries

llm/config (tiny→Llama-13B), tokenizer (char/word/BPE), embedding (RoPE), attention (multi-head + KV cache), transformer (blocks), generate (greedy/top-k/p/beam), train (loops/LR sched), agent (agentic framework), prompt (ChatML/Llama/Alpaca), lora (adapters), quantize (int8/int4), engram (neural memory), rag, dpo, gguf (export for Ollama/llama.cpp), gguf_import (Ollama→SageGPT), turboquant (3-bit KV, 8.5x, ICLR 2026), autoresearch (Karpathy ratchet), evolve (self-evolving architectures).

9.17 Agent Framework (lib/agent/)

agent/core (ReAct loop), tools (file/code/search), planner (DAG decomposition), router (multi-agent), supervisor (Supervisor-Worker), critic (verification loops), schema (typed tools), trace (SFT), grammar (constrained decoding), sandbox (sandboxed Sage exec — hardened is_safe_command), tot (Tree-of-Thoughts + MCTS), semantic_router (fast dispatch).

9.18 Chatbot Framework (lib/chat/)

chat/*: personas, backends bridging llm/*, adapter for Discord and CLI chat; models served by SageGPT chatbot (make chatbot-c/llvm/native).

9.19 UI Widget Library (lib/graphics/ui.sage)

Immediate-mode GPU UI (Dear ImGui style): window/panel/button/label/checkbox/slider/scrollbar/menu/text-input/progress/separator/tooltip; 20+ theme colors + sizes; mouse/keyboard tracking; draw-list batched for Vulkan/GL. API: ui_create(), ui_begin_frame(ctx), widget calls, ui_end_frame(ctx), ui_render(ctx, cmd, font).

9.20 AVR Assembler & Arduino Uno Support (core/boards/AVR/)

Three modules: avr_assembler.sage (two-pass with label resolution + branch offsets + I/O names), avr_opcodes.sage (bit-exact encoder verified vs datasheet + simavr decoder), avr_hex.sage (Intel HEX emitter → .hex). Example blink; smoke test. Target --target avr / atmega328p / atmega328pb.

9.21 Hardware Natives (Embedded Targets) — metal.* v4.2.2

metal.core (hardened in 4.2.2): dmb/dsb/isb/fence memory barriers, cpu relax hints, cpu_id query, critical_section_enter/exit, spin locks (alongside metal irq double-reg guards). metal.vga (4.2.1→4.2.2): text-mode make_attr, putchar_at, puts, read_char_at, read_attr_at, clear, draw_progress_bar. metal.timer: mode state timer_get_mode. Shared with host via sgpu_*/runtime helpers where applicable; baremetal builds use freestanding entry.

All lib/ imports respect SAGE_MAX_READ_SIZE and sandbox guards; experimental broken sources both-host parse errors: os.boot.dtb, os.boot.elf_load, os.ext, transpiler.json_parser, transpiler.python.{ast_parser,emitter,factory} — need source fixes, not interpreter changes.

Part 10 Native Standard Library Modules (Phase 11)

10.1 Math • 10.2 IO • 10.3 String • 10.4 Sys • 10.5 FAT

Native modules pre-registered via create_native_module() and cached: math (sqrt/sin/cos/tan/floor/ceil/abs/pow/log/pi/e/inf/tau 25+), io (readfile/writefile/appendfile/exists/remove/rename/isdir/filesize/readbytes/writebytes/appendbytes/listdir/mkdir — io.* GC-rooted via string interner), string (find/rfind/startswith/endswith/contains/char_at/repeat/count/substr/reverse), sys (args/exit/platform/version/env/setenv/clock/sleep), fat (FAT12/16/32 boot sector parsing). Call forms: import math; math.sqrt(16); or from math import sqrt, pi.

See CLI_Reference for --compile-pico/bare/uefi syscalls; Library_Support matrix for live import verification.

Part 11 Concurrency and Async/Await (Phase 11, hardened 4.2.2)

11.1 Thread Module

```
import thread
let t = thread.spawn(my_proc, args) # pre-evaluated args
print thread.join(t)
let m = thread.mutex(); thread.lock(m); # critical; thread.unlock(m)
thread.sleep(500)
print thread.id()
# cross-platform stubs on RP2040 (pthread vs minimal)
```

11.2 Async/Await

```
async proc compute(x): return x*x
let fut = compute(42) # spawns background thread via thread_spawn_native
print await fut      # joins thread, returns value (EXPR_AWAIT)
# JIT/AOT pragmas: @nojit / @noaot / @noprofile to exclude profiling
```

11.3 GC Thread Safety

GC mutex + TLS root registry + env_mutex protect mark/sweep; barrier flag shared. Concurrency primitives cross-platform: RP2040 fast-path stubs keep API surface identical.

11.4 Synchronization Primitives — now includes rwlock (4.2.2)

Primitive	C API	High-level (import)	Use
Mutex	thread.mutex()/lock/unlock	—	Exclusive lock
Atomics (true __atomic)	atomic_new/load/store/add/cas/exchange (+ sub/fetch and/or)	import std.atomic	Ints/flags/spinlocks
Semaphores	sem_new/wait/post/trywait	import std.condvar (sem)	Counting, POSIX
Condvars	sage_cond_wait/signal/broadcast	import std.condvar	Barriers/latches; C
RW Locks (new)	sage_rwlock_rdlock/wrlock (C VM)	import std_rwlock create()/read_lock/unlock/write_lock/unlock/try_ + with_* helpers import os.sync_rwlock *	Many readers vs single writer
Channels	—	import std.channel (Go-style buffered)	Buffered channel
Threadpool	—	import std.threadpool (parallel map)	Work queue / futures
SMP detect	cpu_count()/cpu_physical_cores()/cpu_has_hyperthreading()	import os.smp topology() etc.	Pinning & work distribution

SMP helpers lib/os/smp.sage: topology(), pin_to_core(id), current_core(), per_cpu_array/get/set, parallel_for_cores(items, fn), on_all_cores(fn), send_to_core(id, fn). Core affinity via thread_set_affinity(core) / thread_get_core().

Part 12 Developer Tooling (Phase 12)

12.1 REPL (Read-Eval-Print Loop)

Command	Meaning
:help	Print help
:quit / :exit / Ctrl-D	Leave session
:reset	Reset session, globals, module cache
:clear	Clear screen
:history [n]	Last n entries (default 20)
:save <file>	Save history to .sage file
:vars [prefix]	List bindings, optional filter
:type <expr>	Runtime type + value
:ast <code>	Parsed AST dump
:env	Full scope chain (parents)
:modules	Loaded modules + search paths
:doc <name>	Docstring for builtin/keyword/user proc

:edit [file]	External editor compose + exec
:ls [dir] / :cat <file> / :sh <cmd>	Directory/file/shell (dash-rejection hardened)
:search <pat> / :clear-history	History search / clear
:emit-c/llvm/kotlin <code>	Show backend output
:stats	GC stats, stack depth, CPU time
:time <expr>	Time single eval
:bench <n> <expr>	n runs, min/avg/max
:load <file> / :pwd / :cd <dir>	Exec file in session / directory ops
:gc / :runtime [mode]	GC + stats / show switch ast bytecode auto

12.2 Code Formatter

```
sage fmt file.sage          # in place -> 'Formatted: file.sage'
sage fmt --check file.sage  # check only, exit 1 if needed
```

12.3 Linter

sage lint file.sage — 13 rules: E001–E003 errors (syntax/structural), W001–W005 warnings (potential bugs), S001–S005 style (incl. S003 missing docstring, S005 multiple statements per line — added v4.1.11). Reports file:line:code:message, exit 1 on issues.

12.4 Syntax Highlighting

TextMate grammar editors/sage.tmLanguage.json (keyword.control.sage, keyword.declaration.sage, keyword.other.sage, keyword.operator.sage, constant.language.sage, entity.name.function/type.sage, string.quoted.double.sage, comment.line.number-sign.sage) + VSCode extension editors/vscode/.

12.5 Language Server Protocol (LSP)

sage --lsp (stdin/stdout) + standalone sage-lsp binary. Features: diagnostics (real-time save), completion (keywords+symbols), hover (type + doc including user-defined procs via shared g_hover_docs in diagnostic.c), formatting (textDocument/formatting). Integrated tests: make test-selfhost-lsp / make test-all.

Part 13 Self-Hosting (Phase 13) — Full Parity (v4.2.0) — 4.2.2 veri

Sage-on-Sage: lexer, parser, interpreter, formatter, linter, LSP, codegen, compiler all ported from C to Sage. Differential harness testsuite/parity/run_parity.sh executes 28 feature cases through three stacks — C interpreter, self-hosted interpreter, self-hosted compiled binaries (--emit-c → gcc) — byte-identical on all three as of v4.2.0 (verified again for 4.2.2 doc sync). See core/docs/meta/PARITY.md for matrix; one-command bootstrap via sagemake --selfhost.

13.1 Architecture

File	Lines	Role
src/sage/token.sage	—	Token type constants (54)
src/sage/ast.sage	—	Dict-based AST constructors
src/sage/lexer.sage	~300	Indent-aware tokenizer, dict keyword lookup
src/sage/parser.sage	~700	Recursive descent, 12 levels, match guards, soft keywords
src/sage/interpreter.sage	~1050	Tree-walking evaluator, dict Value representation, TLS stack guard
src/sage/sage.sage	—	Bootstrap entry point
src/sage/typecheck.sage	228	Declared/inferred tracking (TypeMap.declared, annotation to kind),
src/sage/formatter/linter/lsp/*	—	Tooling ports
src/sage/codegen + compiler.sage	—	Emit paths shared with C host

13.2 Running Self-Hosted Code

```
cd src/sage && ../../sage sage.sage program.sage
make sage-boot FILE=examples/hello.sage
make test-selfhost          # all 178 bootstrap suites
make test-all              # 2070+ (331 interpreter + 28 compiler + 88 JSON + 1623 self-hosted)
```

13.3 Key Design Decisions (parity fixes landed)

Dict-based Value/AST representation (no C structs); short-circuit and/or emitted as boolean short-circuit; string slice negative wrap; bytes API array-backed; type() indexof contains chr ord int; dict for-in insertion-order keys; default params filled at call sites; comptime lets collected as globals; match guards stored per clause; diagnostics normalized to C (error: severity, no doubled prefix); property miss → stderr + nil (not raise); divergence fixed: elif-chaining, dict print key quotes, val_tag handler, defer LIFO partitioning, division/mod by zero catchable, closure heap-env promotion with two-pass hoisting for nested named procs, eager generator collection (SAGE_TAG_GENERATOR). GC disabled for self-hosted code (gc_disable()).

13.4 Feature Coverage (4.2.2)

Arithmetic, variables, control flow, functions/recursion/closures, classes/inheritance, arrays/dicts/strings, try/catch/break/continue, bitwise ~ via -(n+1), module imports (multi-path + __init__.sage), type annotations (let x: Int / proc f(x: Int): Int), soft keywords (match/init/enum/struct/trait/end as identifiers), loop iteration limits 1M, array_extend, hybrid JIT/AOT profiling (always-on specialization with no flags — hot 50+ calls, monomorphic number fast path, loop specialization helpers; see JIT_AOT_Guide.md). Self-hosted test totals after audit: 3 failing assertions vs 27 at baseline (pre-audit, tracked separately).

13.5 Test Suites

```
make test-selfhost-lexer (12)
make test-selfhost-parser (130)
make test-selfhost-interpreter (18)
make test-selfhost-bootstrap (18)
make test-selfhost-formatter / -linter / -value / -pass / -constfold / -dce / -inline / -typecheck / -stdlib / -module / -llvm-ba
# CMake: cmake -B build -DBUILD_SAGE=ON && cmake --build build --target test_selfhost
```

Note: parity report file core/docs/meta/PARITY.md records dynamic verdict: PARITY (all three stacks) 28/28 — FULL PARITY. Previous static matrices (22/28 SHI) are historical; they are superseded by the dynamic 28/28 harness result reproduced for v4.2.2.

Part 14 Networking (Phase 14)

14.1 Socket Module (low-level POSIX)

```
import socket
socket.create(socket.AF_INET, socket.SOCK_STREAM, socket.IPPROTO_TCP)
socket.bind(fd, "0.0.0.0", 8080); socket.listen(fd, 16)
let cli = socket.accept(fd)
socket.send(cli, "hello"); let data = socket.recv(cli, 4096)
socket.sendto(fd, data, "1.2.3.4", 9000); socket.recvfrom(fd, 4096)
socket.setopt(fd, level, opt, val); socket.poll(fds, timeout)
socket.resolve("example.com"); socket.nonblock(fd); socket.close(fd)
```

14.2 TCP Module (high-level)

```
import tcp
let srv = tcp.listen(9000); let conn = tcp.accept(srv)
let cli = tcp.connect("127.0.0.1", 9000)
tcp.send(conn, data); tcp.recv(conn, 4096)
tcp.sendall(conn, data); tcp.recvall(conn, size); tcp.recvline(conn)
tcp.close(conn)
```

14.3 HTTP Module (libcurl)

```
import http
let r = http.get("https://example.com", {"timeout": 5000})
# r == {status: 200, body: "...", headers: {...}}
http.post(url, data, opts); http.put(url, data); http.delete(url)
http.patch(url, data); http.head(url); http.download(url, filepath)
http.escape(s); http.unescape(s)
```

14.4 SSL Module (OpenSSL)

```
import ssl
let ctx = ssl.context()
ssl.load_cert(ctx, cert_path, key_path)
let h = ssl.wrap(ctx, fd); ssl.connect(h, host, port); ssl.accept(h)
ssl.send(h, data); ssl.recv(h, 4096); ssl.shutdown(h); ssl.free(h); ssl.free_context(ctx)
ssl.error(); ssl.peer_cert(h); ssl.set_verify(ctx, mode)
```

14.5 Important Notes

Desktop link links libcurl + OpenSSL via pkg-config in Make/CMake; headless without them still builds (stubbed modules). 100MB SAGE_MAX_READ_SIZE enforced on tcp.recv. See Networking_Guide.md for native module table and pure-Sage companions net.url/headers/request/server/websocket/mime/dns/ip.

Part 15 JSON Library (cJSON Port)

Complete 1:1 port of Dave Gamble's cJSON in lib/json.sage (~1050 lines), plus Sage extras. 88 tests passing. Verified against C boundary tests bridged from tests/12_json.

15.1 Basic Usage

```
import json
let parsed = json.cJSON_Parse("{\"name\": \"Alice\", \"age\": 30}")
print json.cJSON_GetObjectItemCaseSensitive(parsed, "name")
print json.cJSON_IsNumber(json.cJSON_GetObjectItem(parsed, "age"))
```

15.2 Creating JSON

```
let arr = json.cJSON_CreateArray()
json.cJSON_AddItemToArray(arr, json.cJSON_CreateString("hello"))
let obj = json.cJSON_CreateObject()
json.cJSON_AddItemToObject(obj, "key", json.cJSON_CreateNumber(42))
# also: CreateNull/True/False/Bool/Number/String/Raw/IntArray/DoubleArray/FloatArray/StringArray
```

15.3 Sage-Native Conversion

```
let native = json.cJSON_ToSage(parsed) # tree -> Sage dict/array
let tree = json.cJSON_FromSage({"a": [1,2,3]}) # Sage -> tree
print json.cJSON_Print(tree) # formatted; PrintUnformatted for compact
```

15.4 API Reference (selection)

Family	Functions
Parsing	cJSON_Parse, ParseWithLength, GetErrorPtr
Printing	cJSON_Print (formatted), PrintUnformatted, Minify, Version
Creation (13)	CreateNull/True/False/Bool/Number/String/Raw/Array/Object/IntArray/DoubleArray/FloatArray/StringArray
Query	GetArraySize, GetArrayItem, GetObjectItem (+CaseSensitive), HasObjectItem, GetStringValue/GetNumberValue
Type checks (10)	IsValid/False/True/Bool/Null/Number/String/Array/Object/Raw
Modification	Array insert/detach/delete/replace; Object add/detach/delete/replace (case-sensitive variants); SetValueString/SetNumberHelper
Utility	Duplicate, Compare, Delete, + Sage extras ToSage/FromSage

15.5 Important Notes

Binary-safe I/O hardened for null bytes (length-aware interning). Fuzz target scripts/fuzzer.py exercises cJSON parser with coverage-guided generation.

GPU Graphics Vulkan + OpenGL (Phase 15-16)

Five-layer engine: Layer5 Sage Demos → Layer4 Engine libs
 (renderer/scene/material/pbr/postprocess/shadows/deferred/taa/gltf/asset_cache/frame_graph/debug_ui/ui/octree/lod/trails)
 → Layer3 Builder libs (graphics/vulkan.sage, graphics/opengl.sage) → Layer2 Execution path (interpreter src/c/graphics.c
 5700 lines | LLVM llvm_runtime.c 103 sage_rt_gpu_* | VM 30 BC_OP_GPU_*) → Layer1 Pure C GPU API gpu_api.h/c (~100
 sgpu_* funcs) → Vulkan SDK / OpenGL 4.5 + GLFW. Handle-table resource model; auto-detect SAGE_HAS_VULKAN /
 SAGE_HAS_OPENGL; stub mode without SDK returns errors gracefully; windowed vs headless supported. Build: make (auto),
 make VULKAN=1 OPENGL=1, make shaders (glslc → .spv).

Build Requirements & Quick Starts

```
make # auto-detect Vulkan+GLFW+OpenGL
make VULKAN=1 OPENGL=1 # force; VULKAN=0 OPENGL=0 to disable
make shaders # compile .vert/.frag/.comp -> .spv
sage --compile-llvm my_game.sage -o my_game # links -lvulkan -lglfw -lGL automatically
# interpreter:
import gpu; gpu.init_windowed("My App", 800, 600, "Game", false)
print gpu.device_name(); gpu.poll_events(); gpu.shutdown_windowed()
# OpenGL alt:
import graphics.opengl; opengl.init_windowed("My App", 800, 600)
```

Quick Start: Empty Window & Triangle (outline)

Create render pass ({format: FORMAT_SWAPCHAIN, load_op: LOAD_CLEAR, final_layout: LAYOUT_PRESENT}), swapchain
 framebuffers, command pool/buffer, semaphores + fence. Render loop: poll_events → wait_fence/reset_fence →
 acquire_next_image(img_sem) → begin_commands → cmd_begin_render_pass → (bind pipeline/descriptors/vertex buffers →
 cmd_draw/cmd_draw_indexed) → cmd_end_render_pass → end_commands → submit_with_sync(cmd, img_sem, rdr_sem,
 fence) → present(idx, rdr_sem) → device_wait_idle → shutdown_windowed. See Vulkan_GPU_Guide.md minimal window +
 renderer.sage high-level path (create_renderer, begin_frame/end_frame, aspect_ratio, check_resize).

GPU Module API Reference (abridged)

Area	Key Functions (selection)
Context	has_vulkan(), has_window, initialize/init_windowed, device_name/limits, last_error, shutdown, get_active_backend
Buffers	create_buffer/size/usage/mem, create_uniform_buffer, update_uniform, buffer_upload/download, upload_device_local, upload_bytes, destroy_buffer
Images	create_image/_3d/cubemap, create_depth_buffer, load_texture, generate_mipmaps, create_sampler/advanced, destroy_image
Shaders/Pipelines	load_shader (+ reload), create_pipeline_layout, create_compute_pipeline, create_graphics_pipeline(cfg dict), destroy_pipeline
Render passes	create_render_pass/_mrt, create_offscreen_target, create_framebuffer, create_swapchain framebuffers(depth)
Commands	begin_commands/begin_secondary/end_commands, cmd_bind_*, cmd_dispatch/_indirect, cmd_push_constants, cmd_begin/end_render_pass, cmd_draw/_indexed/_indirect, cmd_bind_vertex/_index_buffer, cmd_set_viewport/scissor, cmd_pipeline/barrier, cmd_execute_commands
Sync	create_fence/wait_fence/reset_fence, create_semaphore, acquire_next_image, present, submit/_with_sync, submit_compute, recreate_swapchain
Input	update_input, key_pressed/_just_pressed/_just_released, mouse_pos/delta/button, scroll_delta, text_input_available/read, set_cursor_mode, get_time, save_screenshot/screenshot

Constants: 100+ Vulkan enums (BUFFER_* 8 values, MEMORY_* 3, FORMAT_* 8+, STAGE_* , etc.) OR-composable; image
 barriers; descriptor types (STORAGE_BUFFER/UNIFORM_BUFFER/SAMPLED_IMAGE/STORAGE_IMAGE/COMBINED_SAMPLER).
 Libraries: graphics.math3d (vec3/mat4/persp/look_at/camera_orbit/fps), graphics.mesh (cube/plane/sphere procedural +
 OBJ + upload_mesh), graphics.camera (interactive FPS/orbit), graphics.renderer (double-buffered frame lifecycle),
 graphics.pbr (Cook-Torrance GGX/Smith/Schlick + Reinhard), graphics.postprocess (HDR/bloom/ACES), graphics.shadows
 (shadow map + cascade), graphics.deferred (G-buffer + SSAO + SSR), graphics.taa (jitter + history), graphics.scene
 (hierarchy), graphics.material (pipeline factory), graphics.asset_cache, graphics.frame_graph, graphics.debug_ui,
 graphics.gltf, plus octree/lod/trails spatial utilities and immediate-mode graphics.ui (see Part 9.19).

Library Support Matrix v4.2.2 — 295 modules (Library_Support.md)

Verified by importing all 295 modules from a neutral working directory. Execution-command import matrix:

Interpreter	Modules importing cleanly	Remaining failures
sage-c (C host)	281 / 295	Contract payloads, osdev-experimental sources, shell-out examples (by design)
sage (self-hosted)	273 / 295	Also lacks enum/trait keywords + some C-only natives (see below)

Fully supported under BOTH interpreters

arrays, assert, chat/*, crypto/*, dicts (hardened), iter, json, math, option, perf, rich/*, stats, std/* (except std.enum/trait on sage), strings, string, sys, utils, transpiler/lily/*, transpiler/base, agent/*, blockchain/* (see exclusions), net/*, android/*, cuda/*, discord/*, gc/*, llm/*, metal/* (metal.vga/metal.core now aligned), mips/*

C-host-only behavior

Module	Reason
ml.gpu_accel, ml (import-time)	Calls ml_native C accelerator during import; sage has importable stub but top-level fails
os.*, linux.syscalls	Uses struct_def/struct_new builtins; sage-c continues past runtime errors, sage raises
net.websocket	Parse error both hosts (pre-existing, not 4.2 regression)

Not importable by design

Module	Reason
blockchain.staking, blockchain.std.nft	Smart-contract payloads expecting injected state binding from contract host
os.examples.*	Demo programs that shell out at import — blocked by sandbox is_safe_command

Known broken sources (parse errors BOTH hosts)

os.boot.dtb, os.boot.elf_load, os.ext, transpiler.json_parser, transpiler.python.{ast_parser,emitter,factory} — experimental modules with syntax current parser rejects; need source-level fixes, not interpreter changes.

Self-hosted (sage) feature gaps (tracking closes)

enum / trait declaration keywords not yet parsed (std.enum, std.trait fail until parser lands). Host-only native modules (gpu, http, tcp, socket, thread, ssl, ffi, vm, ml_native) import as empty stubs on sage; calling raises missing-attribute error. Gradually shrinking as self-hosted typecheck/codegen coverage expands (proc type annotations closed in 4.1.13-14).

Appendix Quick Reference (from SageLang_Reference.md v4.1.14)

Keywords (47 total, incl. soft)

and as async await break case catch class comptime continue default defer elif else end enum false finally for from if import in init let macro match nil not or print proc quote raise return self struct super trait true try unquote unsafe var while yield @ — soft: match, init, enum, struct, trait, end, print (also usable as variable/property/method names). Doc comments ## stored as TOKEN_DOC_COMMENT.

Built-in Functions (selection — 110+ total, see SageLang_Reference.md:381)

Family	Selection
I/O & conv	print, input()->String, clock()->Number, tonumber(s), int(v), str(v), type(v)->String, chr(n)->String, ord(s)->Number
Collections	len(v), push(arr,val), pop(arr), range(end)/range(s,e,step), slice(arr,s,e), array_reverse/contains/index_of/min/max/sum/product/extend/repeat, dict_keys/values/has/delete, hasattr(obj.name), val tag(v).
String	split/join/replace, string_count, string_repeat, upper/lower/strip, startswith/endswith/contains/index_of
GC	gc_collect, gc_stats()->Dict, gc_collections()->Number, gc_enable/disable, gc_mode()->String, gc_set_arc/orc
Bytes	bytes(data.len)->Bytes, bytes_len/get/set/to_string/slice/push

Struct	struct_def(fields)->Type, struct_new(type), struct_get/set, struct_size
Memory/FFI	mem_alloc/free/read/write/size/offsetof/offsetof_raw/sizeof/ptr_add /ptr to int, ffi_open/close/call/svm
Concurrency	cpu_count, cpu_physical_cores, cpu_has_hyperthreading, thread_set_affinity/get_core, atomic_new/load/store/add/cas/exchange, sem_new/wait/post/trvwait
Crypto/misc	hash(v) FNV-1a, sha256, doc(fn), next(gen)
Path/Net	path_join/dirname/basename/ext/exists/is_dir/is_file; socket.create/bind/listen/accept/connect/send/recv/...; tcp.connect/listen/accept/send/recv/sendall/recvll/recvline; http.get/post/...; ssl.context/wrap/connect/...
VM	vm_gas_limit_set/get, vm_gas_used_get; build_quad_verts, build_line_quads; asm_exec/compile/arch; comptime blocks/quote/unquote/macros

Operators (12 precedence levels, left unless noted)

1 or (L) | 2 and (L) | 3 == != (L) | 4 < > <= >= in (L) | 5 | (L) | 6 ^ (L) | 7 & (L) | 8 << >> (L) | 9 + - (L) | 10 * / % (L) | 11 unary - ~ not (R) | 12 . -> [] () (L). Shift validated 0-63 else 0; right-shift arithmetic (sign-extending); floats trunc to long long before bitwise.

CLI Quick Reference (CLI_Reference.md)

```
sage                # REPL (also sage --repl)
sage file.sage [args]    # run; args via sys.args()
sage -c "source"        # run source string
sage file.sgvml         # run SGVM binary via MetalVM
sage fmt file.sage / fmt --check # formatter
sage lint file.sage / check file # linter / type-check
sage --lsp              # LSP on stdin/stdout (sage-lsp companion)
sage --emit-c file.sage -o out.c [-00-3 -g] # C backend
sage --compile file.sage -o bin [--cc cc -00-3 -g] # compile via C
sage --emit-llvm file.sage -o out.ll # LLVM IR
sage --compile-llvm file.sage -o bin # LLVM -> clang link
sage --emit-asm file.sage --target x86-64|aarch64|rv64|mips|avr[-profile]
sage --compile-native file.sage --target arch[-profile]
sage --emit-vm file.sage / --sgvm file.sage # bytecode
sage --emit-kotlin file.sage / --compile-android file.sage [--package ...]
sage --jit file.sage [-o bin] / --aot file.sage -o bin / --aot --jit file.sage -o bin
sage --compile-pico file.sage -o dir [--board pico --chip rp2040]
sage --compile-bare file.sage -o prog.elf [--target x86_64]
sage --compile-uefi file.sage -o prog.efi
sage --gc:arc|orc|tracing file.sage; sage --jit --verbose file.sage
```

Grammar (informal)

program = statement*; block = NEWLINE INDENT statement+ DEDENT; let_stmt = let IDENTIFIER (: type_annotation)? = expr; if_stmt = if expr : block (elif expr : block)* (else : block)? ; while/for/proc/return/break/continue/class(...)/struct/enum/trait/match/defer/try-raise/yield/import(async) / comptime/macro; primary = NUMBER|STRING|BOOL|NIL|IDENTIFIER | [array] | {dict} | (tuple) | (expr) | primary (args) | primary [expr] | primary [expr : expr] | primary . ID | primary -> ID | await expr | super . ID (args) | comptime (expr) ; type_annotation = dotted_name ([type_annotation (, type_annotation)*])? (?)? . Full 12-level precedence parser enforces soft keywords where decls expect them.

This PDF was regenerated August 31, 2026 for v4.2.2 from live markdown. Security: resource limits (100MB SAGE_MAX_READ_SIZE, 1M recursion/loop caps, stack_danger TLS guard), OOM abort-on-failure, path-containment, shell-injection guards, FFI bounds, shift/offset validation, binary-safe I/O, algorithmic complexity fixes (CWE-400/190/78/377).

Resources

Repository: github.com/Night-Traders-Dev/SageLang • Docs: [core/docs/](#) • Book: [core/docs/sagelang-book.md](#) • Reference: [core/docs/SageLang_Reference.md](#) • Roadmap: [core/docs/meta/ROADMAP.md](#) • Changelog: [core/docs/meta/CHANGELOG.md](#) • Parity Report: [core/docs/meta/PARITY.md](#) • Benchmarks: [core/docs/meta/BENCHMARKS.md](#) • Security: [core/docs/meta/SECURITY.md](#) • Benchmark runners: `make benchmark-python`, `make benchmark-python-md`, `bash benchmarks/run_backend_compare.sh`